Keywords: mechanistic interpretability • coding agents • probing • latent representations

Latent Programming Horizons

Coding Agents Know the Outcome of Their Edits Up to 25 Steps Before They Make Them

By André Silva, Han Tu & Martin Monperrus (KTH Royal Institute of Technology, Stockholm)

arXiv:2607.05188 • July 6, 2026

When a language model acts as a coding agent, it spends dozens of steps reading files, reasoning about bugs, writing patches, and running tests—a process that is opaque from the outside. This paper opens that black box. Using logistic-regression probes on the residual streams of GPT-4o and Claude 3.7 Sonnet, the authors show that the model's internal representations linearly encode the correctness of the program being edited with AUC up to 0.83—and that these representations run up to 25 steps *ahead* of the agent's own actions, constituting what the authors call a **latent programming horizon**.

AT A GLANCE

Problem: The internal states of coding agents during multi-step repair episodes are not understood; it is unknown whether models plan ahead or react greedily.

Why it matters: A latent horizon implies agents have exploitable internal signals for early stopping, intervention, and lookahead guidance without external execution.

Method: Logistic-regression probes on final-layer hidden states recorded at each step of coding-agent episodes; future probes predict outcomes k steps ahead.

Result: AUC 0.83 for correctness at step t; horizon k^* up to 25 steps on SWE-bench Verified.

Evidence: Two models (GPT-4o, Claude 3.7), two benchmarks (SWE-bench Verified, HumanEval-Pro); cross-benchmark transfer retains AUC > 0.70.

The Big Picture

Large language models acting as software engineering agents can resolve real GitHub issues, pass coding interviews, and complete complex multi-file refactors—but the mechanism by which they succeed or fail across a long episode remains opaque. Mechanistic interpretability research has largely focused on single forward passes or small fixed-context tasks.

Coding-agent episodes are different: the model sees an evolving repository state, modifies files, receives test outputs, and makes new decisions over potentially hundreds of steps. Understanding what the model "knows" at each step has direct practical value—if the model's internal state already encodes the likely outcome of the episode, that signal could be used to guide, interrupt, or improve the agent without waiting for external test execution.

This paper introduces the first systematic mechanistic interpretability study of coding agents over full multi-step episodes.

Why Existing Approaches Fall Short

Prior mechanistic interpretability work probes fixed-context forward passes (e.g., factual recall, in-context learning, arithmetic). These methods cannot be applied directly to coding-agent episodes because (1) the context evolves across steps as files are modified, (2) ground truth labels depend on external execution of the evolving codebase, and (3) the episode length (up to 200 steps) is far longer than tasks previously studied.

Behavioral analysis of coding agents (e.g., measuring pass@k or success rate) describes what agents do but not what they represent. Existing evaluation frameworks do not distinguish between agents that reason correctly but act incorrectly and agents that reason incorrectly and happen to succeed by luck—a distinction this paper makes tractable via probing.

Core Mechanism

The latent programming horizon arises because the model processes the full current repository state and reasoning history at each step, and its internal representations must encode relevant aspects of the program's state to generate useful next-action tokens.

The key empirical finding is that correctness-relevant features are not just encoded in the final step before an edit—they are stably encoded up to 25 steps earlier in the episode. This suggests the model is tracking a "mental model" of the program's correctness that is both accurate and temporally extended.

Technical Formulation

At each step t of an episode, the last-position final-layer residual stream vector $h_t \in \mathbb{R}^{4096}$ is recorded. Binary labels $y_t \in \{0, 1\}$ are obtained by running pytest on the repository state after each agent edit (parsed: 1/0; test-pass: 1/0; etc.).

For immediate probes, a logistic regression $f_\theta(h_t) \to \hat{y}_t$ is trained on (h_t, y_t) pairs.

For future probes at horizon k, the same probe structure predicts y_{t+k} from h_t. The latent programming horizon

is:

$$k^* = \max\{k \ : \ \mathrm{AUC}(f_{t\to t+k}) \geq 0.55,\ p < 0.05\}$$

where significance is assessed by paired permutation test over episodes.

Learning or Inference Procedure

1. Run coding agents on SWE-bench Verified (300 issues) and HumanEval-Pro (150 problems) with step-by-step residual stream logging enabled via API hooks.
2. After each edit, execute sandboxed pytest and record binary labels for four tasks (parses, passes tests, reduces failures, introduces regression).
3. Train logistic-regression probes on 80% of episodes; evaluate on 20% held-out episodes.
4. Sweep horizon k from 0 to 40; measure AUC and p-value at each horizon to determine k^*.
5. Perform cross-benchmark transfer: train on SWE-bench, evaluate on HumanEval-Pro without retraining.

What the Guarantee Says

The core finding is not a guarantee in the theoretical sense, but a robust empirical regularity: across two models and two benchmarks, the latent programming horizon k^* is consistently 20–28 steps for "passes tests" and 12–16 steps for "introduces regression." Probes trained on one model do not transfer to the other, indicating the representations are model-specific even if the phenomenon is model-agnostic. Cross-benchmark transfer holds above AUC 0.70 for all tasks, confirming the probe captures generic code-quality structure.

Experimental Findings

Task	Model	AUC (k=0)	Horizon k^*
Parses	GPT-4o	0.81	28
Passes tests	GPT-4o	0.78	22
Reduces fails	Claude 3.7	0.83	25
Introduces reg.	Claude 3.7	0.71	14

Cross-benchmark transfer (SWE-bench → HumanEval-Pro) retains AUC above 0.70 for all tasks. Middle-layer probes (layer 24/48) achieve only AUC 0.63, indicating the relevant representations are built through the full network depth. Non-linear probes improve AUC by only 0.02, confirming approximate linear decodability.

Ablations and Interpretation

Layer depth: Probing earlier layers degrades AUC monotonically from 0.83 (final layer) to 0.51 (first layer), suggesting correctness representations are progressively constructed across transformer depth.

Non-linearity: Adding one MLP hidden layer improves AUC by 0.02 on average. The dominant signal is linear, consistent with the probing literature on factual representations.

Shuffled-label control: Probes trained on permuted labels achieve AUC 0.50 ± 0.01, ruling out statistical artifacts from episode length or context size.

Practical implication: The 25-step horizon means an external monitor could identify likely-failing episodes 25 steps early and redirect the agent—a potential 25-step saving per failed episode. On SWE-bench Verified, where 40% of episodes fail, this would reduce total token cost by ∼10%.

Reference

André Silva, Han Tu, Martin Monperrus. **Latent Programming Horizons in Coding Agents**. arXiv:2607.05188, July 2026. KTH Royal Institute of Technology. https://arxiv.org/abs/2607.05188